

Raw-Python vs. LangChain Agent

Comparison of Raw-Python and LangChain Agents

In Day 1, the agent was implemented from scratch using raw Python. The implementation required manually managing message history, defining tool schemas, sending requests to the model, detecting tool calls, executing the selected tools, returning tool results, and repeating the Reason → Act → Observe cycle until the model produced a final answer. This approach made the underlying agent architecture highly visible because each stage of the process was explicitly implemented and logged.

In Day 2, the same basic agent concept was rebuilt using LangChain. The `@tool` decorator replaced the manually written tool schemas, `AgentExecutor` replaced the manually implemented agent loop, and `RunnableWithMessageHistory` handled conversation history across multiple turns. LCEL also provided a concise way to compose fixed pipelines such as `prompt | llm | parser`. These abstractions significantly reduced the amount of orchestration code required while preserving the same general agent workflow.

Annotated Agent Trace

For the multi-tool test, the user asked: "What's the weather in Lahore, and what's the price of Laptop A? Tell me both."

```
[START] > Entering new AgentExecutor
chain...

[ACT]
Invoking: get_weather with {'city': 'Lahore'} The model selected the weather

tool and supplied the required city argument.

[OBSERVE]
{"temp_c": 41, "condition": "sunny"} The weather

tool returned its result to the agent.

[ACT]
Invoking: get_product_price with {'product': 'Laptop A'} The model

selected the product-price tool and supplied the product name.

[OBSERVE]
{"price": 899.0, "specs": "16GB RAM, 512GB SSD, mid-range CPU"} The

product tool returned the price and specifications.

[FINAL]
The final natural-language response The agent combined both

observations into the final response.

[END] > Finished
chain.

The AgentExecutor completed the run.
```

Final response

The weather in Lahore is sunny with a temperature of 41 degrees Celsius. The price of Laptop A is \$899, and it comes with 16GB RAM, 512GB SSD, and a mid-range CPU.

What Changed in LangChain?

The biggest difference is where the orchestration logic lives. In Day 1, the developer explicitly wrote the loop and controlled the tool-call process. In Day 2, LangChain's `AgentExecutor` manages that execution. Similarly, the `@tool` decorator generates tool definitions from function metadata instead of requiring manually constructed schemas.

Conversation history is also easier to manage. In Day 1, messages were manually appended to a growing list. In Day 2, `RunnableWithMessageHistory` maintains the conversation history and makes it available to subsequent turns. This allowed the agent to understand references such as “it” in “compare it to Laptop B” and “which one” in the final recommendation question.

LangChain also provided structured output through Pydantic. The structured-output chain used the product information retrieved by the tools and returned a validated `Recommendation` model containing the product name, price, and reason. This provides a predictable structure instead of relying only on free-form text.

What Became Hidden?

The main trade-off is visibility. The raw-Python implementation exposed the message structure, tool calls, observations, and loop-control logic directly. LangChain hides much of this behind its abstractions. With `verbose=True`, tool invocations and their results are visible, but the model's private reasoning is not separately displayed.

This means LangChain makes development faster and reduces repetitive code, but debugging can require understanding the framework's abstractions. Settings such as iteration limits and tool behavior are configured through framework components rather than being implemented directly in the agent loop.

The error-handling experiment also demonstrated this abstraction trade-off. An unhandled tool exception stopped the agent, while the resilient version used tool-level `try/except` handling to convert the failure into a controlled `TOOL_ERROR` observation. The agent then used that observation to produce a graceful response without inventing product information.

Conclusion

The Day 1 implementation demonstrated what an agent actually does internally, while Day 2 demonstrated how a framework packages that functionality into reusable abstractions. LangChain made tools, agent execution, conversation history, structured output, and error recovery easier to implement. However, the raw-Python version provided greater visibility into the underlying process.

Building the agent manually first therefore makes the LangChain abstractions easier to understand rather than treating them as “magic.” The main lesson is that frameworks reduce repetitive implementation work, but developers still need to understand the abstractions and execution behavior underneath them.